

Month 2 — Deep Learning for Metal Part Defect Detection

(Builds on Month 1 OpenCV foundations. Difficulty ramps up: from classic CV into applied deep learning, with everything oriented toward the end goal — an automated metal product quality inspection system.)

Week 5 — Deep Learning Foundations & Dataset Building

- CNN basics (conv/pool/fc layers, why CNNs beat manual feature engineering for defects)
- Transfer learning with pretrained backbones (ResNet18/MobileNetV2)
- Building an industrial dataset: capturing consistent lighting/angle for metal parts, labeling tools (LabelImg, CVAT, or Roboflow), class imbalance (defects are rare)
- Data augmentation tuned for metal surfaces: brightness/glare variation, rotation, slight blur (simulate line-speed motion), synthetic scratch/rust overlay
- Task: Train a binary classifier (Good / Defective) on a small metal-parts image set using transfer learning; report accuracy, precision, recall (not just accuracy — defect misses matter more than false alarms)

Week 6 — Custom Object Detection with YOLO

- YOLOv8 architecture overview (why detection > classification when defect location/type matters)
- Annotating bounding boxes for defect types: scratch, dent, rust, crack, burr
- Training a custom YOLOv8 model on the metal-parts dataset, transfer learning from COCO weights
- Evaluation: IoU, mAP@0.5, precision/recall curves, confusion between defect classes
- Task: Train YOLOv8 to detect and label defect type + location on metal part images (use a public metal-surface defect dataset — e.g. NEU-DET or GC10-DET — if real factory data isn't ready yet)

Week 7 — Segmentation, Measurement & Anomaly Detection

- Semantic vs instance segmentation; intro to YOLOv8-seg / U-Net
- From detection to measurement: defect area, length, severity scoring using masks (a 2mm scratch vs a 20mm scratch should not be treated the same)
- Anomaly detection for rare/unseen defects (autoencoders, or lighter methods like PaDiM/PatchCore) — critical in real inspection lines where defective samples are scarce and you can't label every failure mode in advance
- Task: Build a "normal vs anomalous" pipeline using an autoencoder's reconstruction error as an anomaly score, tested against parts with defect types not seen in training

Week 8 — Pipeline Integration, Optimization & Deployment

- Model export/optimization: ONNX conversion, quantization for faster inference
- Real-time inference on video/camera feed, frame-rate considerations for a

production line

- Building the full pipeline: camera capture → preprocessing → detect/segment → pass/fail decision → logging + alert (e.g. flag part ID, defect type, severity)
- Intro to deployment targets: edge devices (Jetson/Raspberry Pi) vs a simple Flask/FastAPI dashboard for a demo setup
- Capstone task (both interns): End-to-end metal part quality inspection demo — takes an image or live video stream, detects/classifies defects, outputs a pass/fail verdict with an annotated visual overlay and a simple summary report

Notes for running Month 2:

- If real factory images of defective parts are limited, start Week 6-7 tasks on a public metal-surface defect dataset (NEU-DET, GC10-DET, or Kaggle's steel defect set) and swap in real data once enough samples are collected.
- Consider having interns keep a shared "defect taxonomy" doc (defect name, visual example, cause) — useful both for labeling consistency and as a deliverable for you.
- Capstone in Week 8 is a good checkpoint to demo to stakeholders — frame it as a working prototype, not a finished product.